Partie N°1 - JavaScript ES6

1. Introduction

JavaScript, langage permettant des interactions faciles entre une page web et l'utilisateur, a connu diverses évolutions ces dernières années, dont la plus significative en **2015** est celle de la version **ES6** (abréviation de **ECMAScript 6**).

ECMAScript est la norme utilisée par le langage de programmation JavaScript. **ECMAScript** fournit la spécification sur le fonctionnement du langage de programmation JavaScript.

La nouvelle version **ES6** concerne particulièrement :

- Les variables : déclaration, portée et mise en forme dans les chaînes de caractères ;
- L'instruction JavaScript for/of ;
- Les fonctions : paramètres par défaut, nouvelle forme de déclaration des fonctions ;
- Les objets et les tableaux : déstructuration et opérateur… ;
- Les classes d'objets : création et dérivation ;
- Les promesses : utilisation du processus asynchrone ;
- Les modules : pour mieux structurer le code JavaScript

React utilise de façon intensive ces nouveaux éléments.

Dans la suite du chapitre, nous utiliserons un fichier index.html qui contiendra le code JavaScript utilisé (au moyen de balises <script>). La structure de ce fichier est la suivante :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>

    <script>
        // Ici, le code JavaScript
        // ...
    </script>
</body>
</html>
```

2. Les variables

Auparavant, les variables étaient déclarées à l'aide du mot clé **var**, depuis la version **ES6**, diverses possibilités de déclaration des variables ont été ajoutés au langage JavaScript tel que l'utilisation du mot clé **const** et **let** :

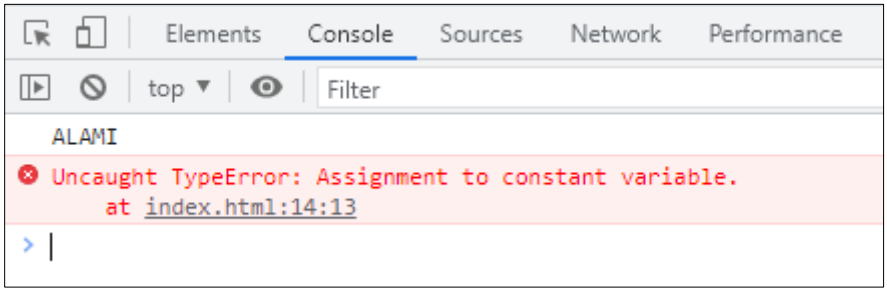
- **var** : Les identificateurs déclarés à l'aide de **var** ont une portée de fonction, sauf lorsqu'ils sont déclarés directement dans le contexte global, auquel cas ils sont ajoutés en tant que propriétés sur l'objet global et ont une portée globale.
- **let** et **const** : Les identifiants déclarés avec **let** et **const** ont une portée de bloc, sauf lorsqu'ils sont déclarés directement dans le contexte global, auquel cas ils ont une portée globale.

Utilisation de const :

Le mot-clé **const** permet de définir une constante qui, par définition, ne pourra plus être modifiée. En cas de modification par le programme, une erreur JavaScript est provoquée.

Exemple 1 :

```
<script>
        const nom = "ALAMI";
        console.log(nom);
        nom = "RAJI"; // Erreur
</script>
```



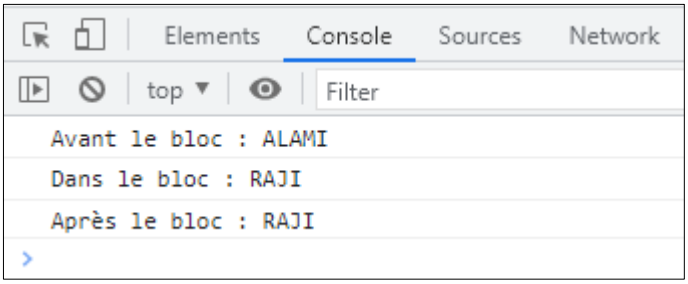
Utilisation de let :

Le mot-clé **let** permet de définir de vraies variables avec une portée de bloc, qui disparaissent lorsque le bloc de code dans lequel elles sont définies n'est plus exécuté. De plus, si une variable du même nom est définie à un niveau supérieur dans le code, cette nouvelle variable définie avec **let** n'écrase pas la valeur de la variable de niveau supérieur. Le comportement de let est bien différent de celui de **var**.

Les deux exemples suivants montrent la différence entre l'utilisation de var et let :

Exemple 2 :

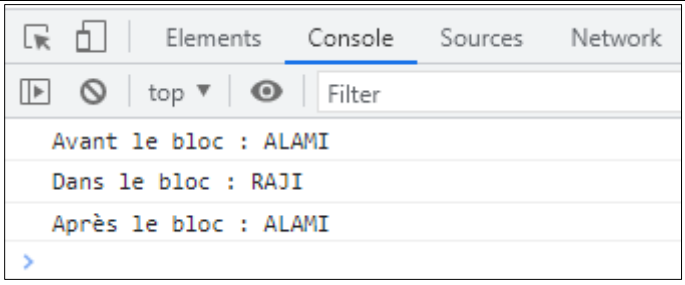
```
<script>
        var nom = "ALAMI";
        console.log("Avant le bloc : " + nom); // "ALAMI"
        if (true) {
            var nom = "RAJI";
            console.log("Dans le bloc : " + nom); // "RAJI"
        }
        console.log("Après le bloc : " + nom); // "RAJI"
</script>
```



L'utilisation du mot-clé **let** va permettre de créer réellement des variables locales dans un bloc, sans interférer avec d'autres variables du même nom définies ailleurs.

Exemple 3 :

```
<script>
        let nom = "ALAMI";
        console.log("Avant le bloc : " + nom); // "ALAMI"
        if (true) {
            let nom = "RAJI";
            console.log("Dans le bloc : " + nom); // "RAJI"
        }
        console.log("Après le bloc : " + nom); // "ALAMI"
</script>
```



3. La boucle for/of

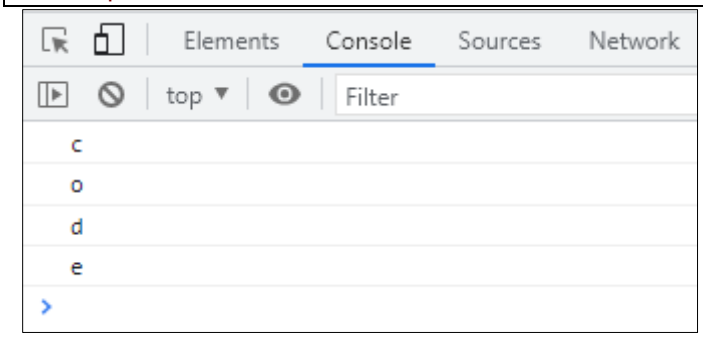
L'instruction JavaScript **for/of** parcourt les valeurs d'un objet itérable. Sa syntaxe est :

```
for (variable of iterable) {
            // code block to be executed
}
```

Voici un exemple avec une chaîne de caractère :

Exemple 1 :

```
<script>
        const string = "code";
        for(let c of string) {
            console.log(c);
        }
</script>
```

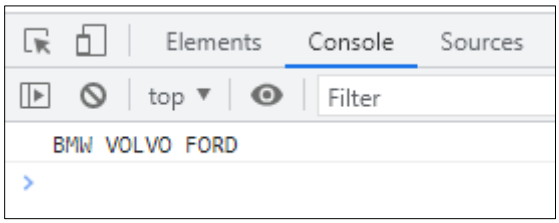


Voici un exemple avec un tableau :

Exemple 2 :

```
<script>
        const cars = ["BMW", "VOLVO", "FORD"];
        let text = "";
        for(let item of cars){
            text += item + " ";
        }
        console.log(text); // "BMW VOLVO FORD"

</script>
```

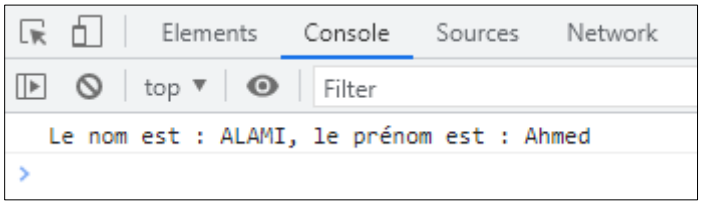


4. Mise en forme des chaînes de caractères

L'utilisation de variables dans une chaîne JavaScript est un peu fastidieuse car il faut fermer chaque chaîne de caractères avant d'utiliser la variable à concaténer à cette chaîne. On écrit souvent des lignes de code telles que celles-ci :

Exemple 1 :

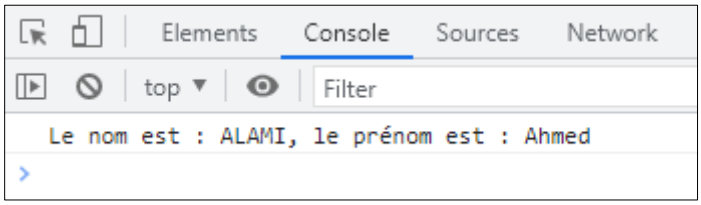
```
<script>
        let nom = "ALAMI";
        let prenom = "Ahmed";
        let txt = "Le nom est : " + nom + ", le prénom est : " + prenom
        console.log(txt);
</script>
```



Une façon plus claire utilisée en **ES6**, sans fermer la chaîne de caractères lors de la concaténation des variables. On utilise pour cela le guillemet simple inversé ` pour définir la chaîne de caractères (au lieu du simple ' ou double "). Chaque variable est utilisée dans la chaîne au moyen de **${variable}** :

Exemple 2 :

```
<script>
        let nom = "ALAMI";
        let prenom = "Ahmed";
        let txt = `Le nom est : ${nom}, le prénom est : ${prenom}`;
        console.log(txt);
</script>
```



5. Les fonctions en ES6

La déclaration des fonctions a été améliorée en ES6, pour la rendre plus performante et plus concise à écrire.

Utilisation de paramètres par défaut :

On va maintenant pouvoir passer des valeurs par défaut à des paramètres de fonction, comme cela se fait dans d'autres langages de programmation.

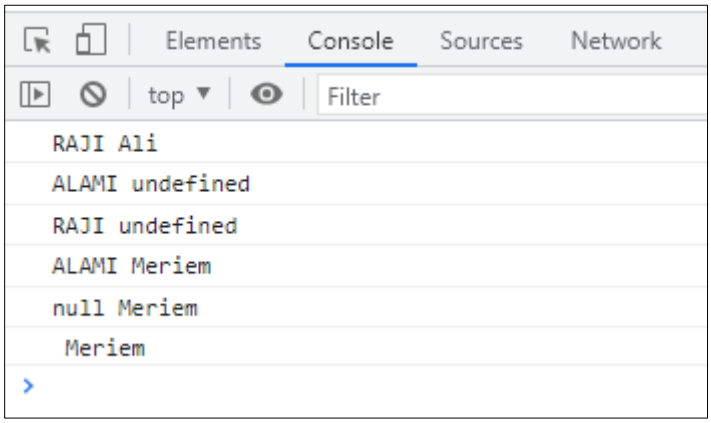
Il faut noter les points suivants :

- Dès qu'une valeur par défaut est indiquée pour un paramètre, tous les autres paramètres qui suivent doivent également avoir une valeur par défaut définie.
- Au moment de l'appel de la fonction, lorsqu'un argument n'est pas utilisé, il est remplacé par sa valeur par défaut.
- Au moment de l'appel de la fonction, il faut utiliser la valeur **undefined** lors de l'appel de la fonction si l'argument n'est pas le dernier (les valeurs **null** ou **""** ne sont pas remplacées par les valeurs par défaut).

Exemple 1 :

```
<script>
        function log(nom = "ALAMI", prenom) {
            console.log(`${nom} ${prenom}`);
        }
        log("RAJI", "Ali");         // "RAJI Ali"
        log();                      // "ALAMI undefined"
        log("RAJI");                // "RAJI undefined"
        log(undefined, "Meriem");   // "ALAMI Meriem"
        log(null, "Meriem");        // "null Meriem"
        log("", "Meriem");          // "Meriem"

</script>
```



Nouvelle forme de déclaration des fonctions en ES6 : (Arrow Functions)

Prenons la fonction **log(nom, prenom)** écrite précédemment. On peut également l'écrire aussi sous la forme suivante :

```
<script>
        let log = function(nom = "ALAMI", prenom) {
            console.log(`${nom} ${prenom}`);
        }
</script>
```

Dans la version **ES6**, le mot-clé **function** a disparu, et il' est remplacé par le signe **=>**. Cette notation s'appelle **Arrow Function** (Fonction fléchée). Les paramètres de la fonction s'écrivent toujours entre parenthèses, tandis que le corps de la fonction est toujours entouré des accolades de début et de fin :

```
<script>
        // Fonction log() écrite en ES6
        let log = (nom = "ALAMI", prenom) => {
            console.log(`${nom} ${prenom}`);
        }
</script>
```

Si la fonction n'a pas de paramètres, on l'écrit de la façon suivante :

```
<script>
        let log = () => {
            return `Bonjour`;
        }
        console.log(log()); // "Bonjour"
</script>
```

Si le bloc de la fonction contient une seule instruction on peut omettre les accolades :

```
<script>
        let log = () => `Bonjour`;
        console.log(log());
    </script>
```

Déclarations de fonction et expressions de fonction

Il y a une différence entre les déclarations de fonction et les expressions de fonction.

Une déclaration de fonction est de la façon suivante :

```
function doSomething () {}
```

Dans une déclaration de fonction, la fonction est hissée entièrement vers le haut de la portée. Cela signifie que les éléments suivants fonctionneront :

```
doSomething() // works!
function doSomething() {}
```

Une expression de fonction est de la façon suivante :

```
[const | let | var] function_name = function () {} (or () =>)
```

Avec une expression de fonction vous avez toujours les règles habituelles concernant la portée des variables ce qui veut dire que la fonction n'est pas hissée vers le haut. Donc les éléments suivants ne fonctionneront pas :

```
doSomething(); // will fail
const doSomething = function () {}
```

Objet this dans les fonctions :

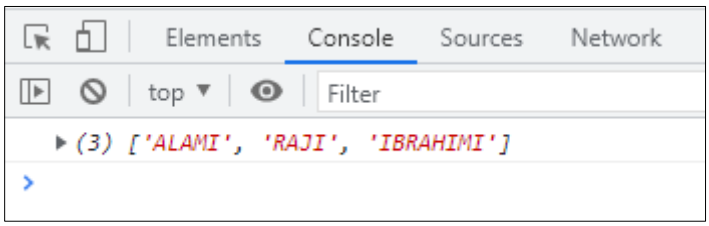
Les **Arrow functions** en **ES6** peut avoir des incidences sur la valeur de l'objet **this**, qui représente l'objet en cours d'utilisation.

Considérons par exemple l'objet suivant, dans lequel est défini une liste de noms à afficher au moyen de la fonction log() définie également dans l'objet. On utilise l'ancienne forme de déclaration des fonctions (avec le mot-clé **function** au lieu de =>).

Exemple 1 :

```
<script>
        // Objet avec fonction log() incorporée (non ES6)
        let obj = {
            noms : ["ALAMI", "RAJI", "IBRAHIMI"],
            log : function() {
                console.log(this.noms);
            }
        };
        obj.log();
</script>
```

L'objet **this** représente ici l'objet en cours d'utilisation de la fonction **log()**, donc l'objet **obj** :

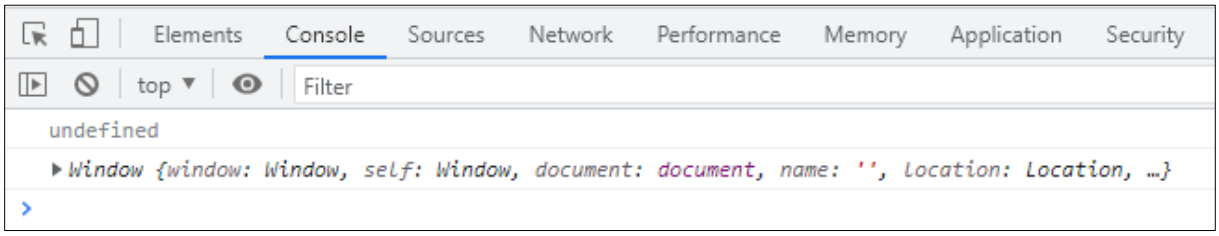


À présent, écrivons la fonction en tenant compte de la notation **ES6** :

Exemple 2 :

```
<script>
        // Objet avec fonction log() incorporée (notation ES6)
        let obj = {
            noms : ["ALAMI", "RAJI", "IBRAHIMI"],
            log : ()=> {
                console.log(this.noms);
                console.log(this);
            }
        };
        obj.log();
</script>
```

La propriété noms de l'objet n'est plus accessible (et vaut donc **undefined**) car **this** représente maintenant l'objet JavaScript **window**, et non plus l'objet **obj** :



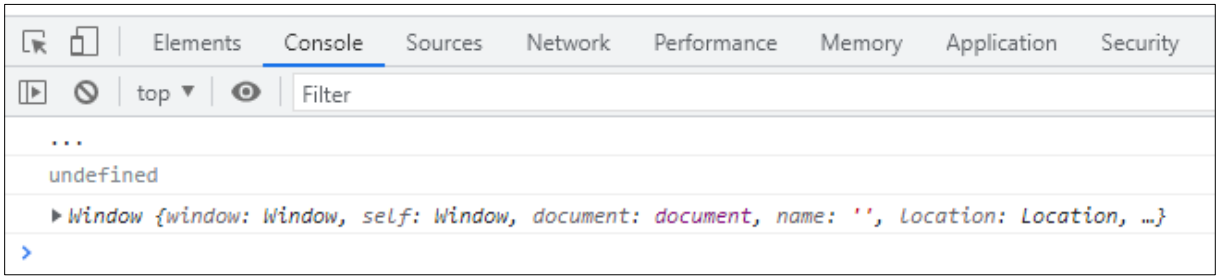
Cette observation peut avoir un intérêt lors de l'utilisation de fonctions asynchrones, très utilisées avec JavaScript.

Considérons que l'on affiche les noms inscrits dans l'objet obj précédent, au bout de quelquesmilli secondes, en utilisant un timer JavaScript. Ce timer s'écrit au moyen d'une fonction asynchrone définie par **setTimeout()** :

Exemple 3 :

```
<script>
        // Utiliser la notation non ES6 pour définir une fonction asynchrone
        let obj = {
            noms : ["ALAMI", "RAJI", "IBRAHIMI"],
            log() {
                setTimeout(function(){
                    console.log(this.noms);
                    console.log(this);
                },5000);
            }
        };
        obj.log();
        console.log("...")
</script>
```

L'objet **this** représente donc l'objet window (c'est-à-dire l'objet qui utilise la fonction **setTimeout()**, comme si nous avions écrit dans le code **window.setTimeout())** :

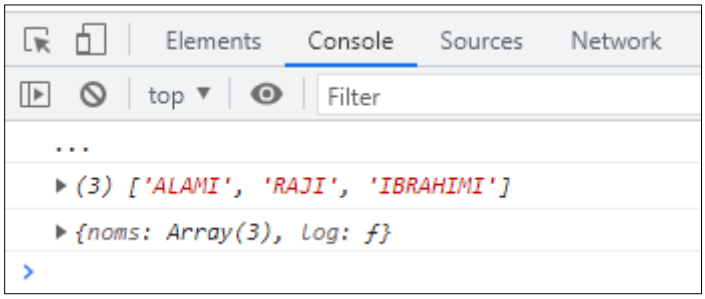


L'utilisation d'une fonction asynchrone modifie complètement le comportement de notre programme. La notation ES6 permet de remédier à ce problème :

Exemple 4 :

```
<script>
        // Utiliser la notation ES6 pour définir une fonction asynchrone
        let obj = {
            noms : ["ALAMI", "RAJI", "IBRAHIMI"],
            log() {
                setTimeout(()=>{
                    console.log(this.noms);
                    console.log(this);
                },5000);
            }
        };
        obj.log();
        console.log("...")
</script>
```

Le résultat est maintenant conforme à nos attentes. La valeur **this** de l'objet est bien conservée et transmise dans la fonction asynchrone :



6. Les objets

Afin de manipuler plus aisément les objets, ES6 permet de les structurer/déstructurer un objet.

Déstructurer un objet :

Déstructurer un objet permet d'accéder aisément aux propriétés d'un objet qui nous intéressent, sans nous préoccuper de celles qui ne nous intéressent pas dans l'immédiat.

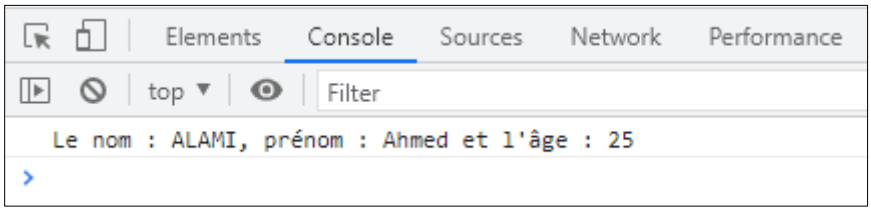
En **ES6**, la destruction d'un objet facilite l'affectation de valeurs des attributs d'un d'objet à des variables distinctes.

Voici des exemples pour comprendre l'utilité de la destruction d'un objet :

Exemple 1 :

```
<script>
        const personne = {
            nom : "ALAMI",
            prenom : "Ahmed",
            sexe : "Homme",
            age : 25
        }
```

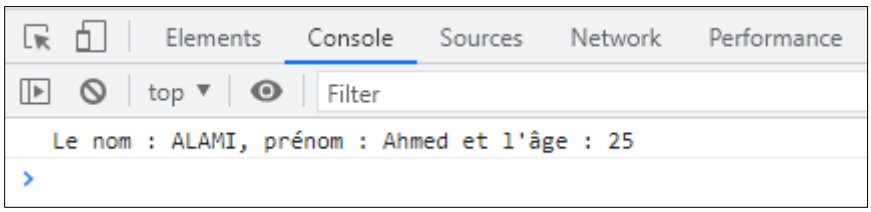
```
        // Sans utiliser la destruction ES6
        let nom = personne.nom;
        let prenom = personne.prenom;
        let age = personne.age;
        console.log(`Le nom : ${nom}, prénom : ${prenom} et l'âge : ${age}`);
</script>
```



Exemple 2 :

```
<script>
        const personne = {
            nom : "ALAMI",
            prenom : "Ahmed",
            sexe : "Homme",
            age : 25
        }
        // Utiliser la destruction ES6
        let {nom, prenom, age } = personne;
        console.log(`Le nom : ${nom}, prénom : ${prenom} et l'âge : ${age}`);
</script>
```

Cela crée trois variables **nom**, **prenom** et **age**, possédant respectivement la valeur de **personne.nom**, **personne.prenom** et **personne.age**. On appelle cela la déstructuration d'un objet, car on accède uniquement aux propriétés intéressantes :



Remarque : Lors de la déstructuration d'objets, vous devez utiliser le même nom pour la variable que la clé d'objet correspondante.

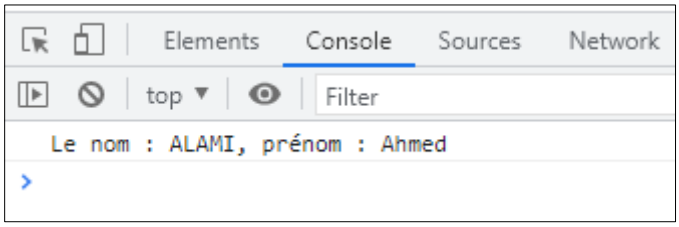
Cette notation sera particulièrement utile pour transmettre un objet en paramètre, en montrant clairement quelles sont les propriétés de l'objet qui sont finalement utilisées dans la fonction. Voici un exemple.

Exemple 3 :

```
<script>
        const personne = {
            nom : "ALAMI",
            prenom : "Ahmed",
            ville : "AGADIR"
        } ;
        let log = ({nom, prenom})=> {
            console.log(`Le nom : ${nom}, prénom : ${prenom}`);
        }

        log(personne);

</script>
```



Voici un autre exemple de la fonction **log()** avec plusieurs paramètres :

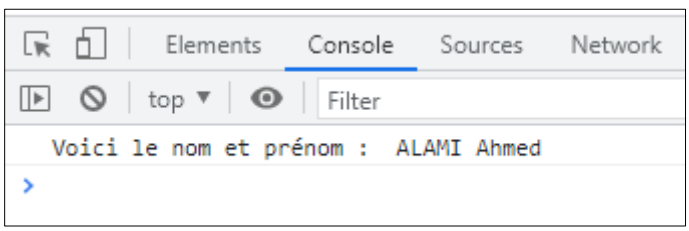
Exemple 4 :

```
<script>
        const personne = {
            nom : "ALAMI",
            prenom : "Ahmed",
            ville : "AGADIR"
        };

        let log = ({nom, prenom}, texte)=> {
            console.log(`${texte} ${nom} ${prenom}`);
        }

        log(personne, "Voici le nom et prénom : ");

</script>
```



Structurer un objet :

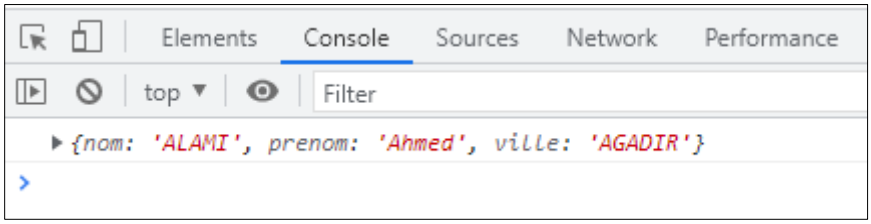
La structuration d'un objet est le processus inverse de la déstructuration, elle permet de créer un objet JavaScript à partir de variables. Voici des exemples :

Exemple 1 :

```
<script>
        let nom = "ALAMI";
        let prenom = "Ahmed";
        let ville = "AGADIR";

        // Sans utiliser la structuration ES6
        const personne = {
            nom : nom,
            prenom : prenom,
            ville : ville
        }

        console.log(personne);
</script>
```

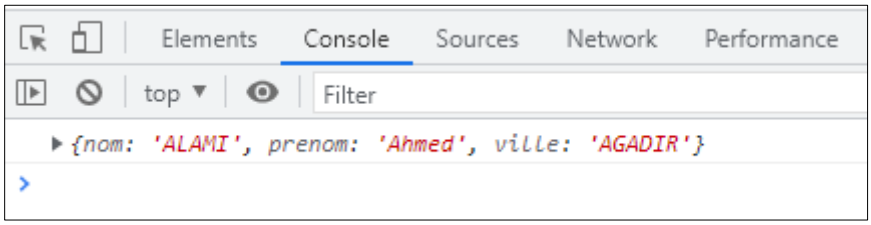


Exemple 2 :

```
<script>
        let nom = "ALAMI";
        let prenom = "Ahmed";
        let ville = "AGADIR";

        // Utiliser la structuration ES6
        const personne = {nom, prenom, ville};

        console.log(personne);
</script>
```



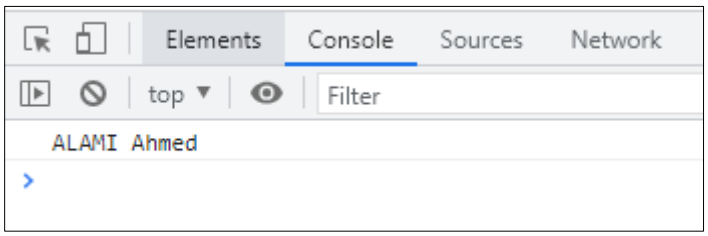
Si une fonction est définie dans les propriétés de l'objet personne, on peut l'écrire de la façon suivante :

Exemple 3 :

```
<script>
        let nom = "ALAMI";
        let prenom = "Ahmed";
        let log = function() {
            console.log(`${this.nom} ${this.prenom}`);
        }

        let personne = {nom, prenom, log};
        personne.log();

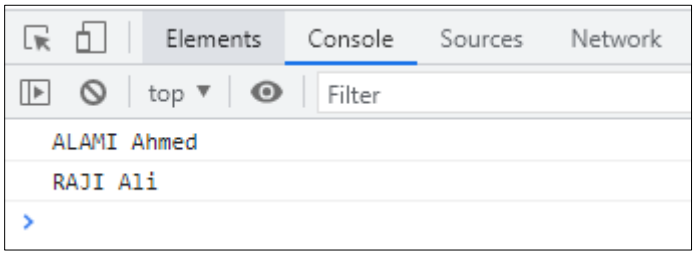
</script>
```



Remarque : ES6 permet de supprimer le mot-clé **function** lors de la définition de la fonction dans l'objet. On peut alors écrire l'objet de la façon suivante :

```
<script>
        // Définition de l'objet p1 sans ES6
        var p1 = {
            nom : "ALAMI",
            prenom : "Ahmed",
            log : function () {
                console.log(`${this.nom} ${this.prenom}`);
            }
        };
        p1.log();
        // Définition de l'objet p2 en ES6
        var p2 = {
            nom : "RAJI",
            prenom : "Ali",
            log() {
                console.log(`${this.nom} ${this.prenom}`);
            }
        };
        p2.log();

</script>
```



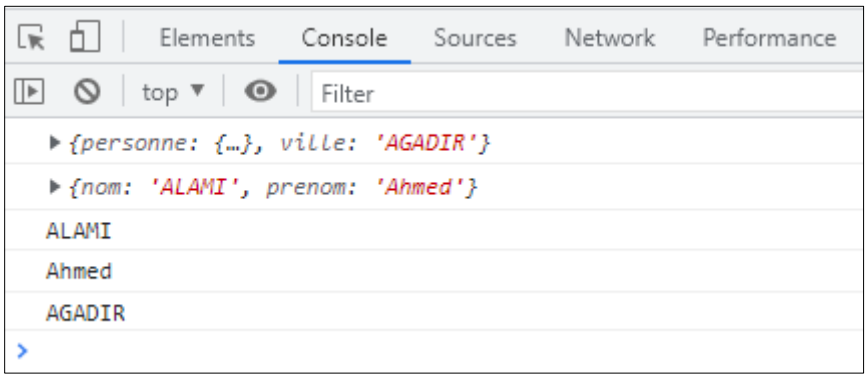
7. Opérateur spread ... sur les objets

L'opérateur **spread** ... (opérateur d'éclatement) permet d'étendre un objet. Par exemple on souhaite créer un nouvel objet **personne2** à partir de l'objet **personne**, mais ce nouvel objet **personne2** devra contenir (en plus) la propriété **ville** (qui n'existe pas dans l'objet personne) :

Exemple 1 :

```
<script>
        let personne = {nom : "ALAMI", prenom : "Ahmed"};
        let personne2 = {
            personne,
            ville : "AGADIR"
        };
        console.log(personne2);
        console.log(personne2.personne);
        console.log(personne2.personne.nom);
        console.log(personne2.personne.prenom);
        console.log(personne2.ville);

</script>
```



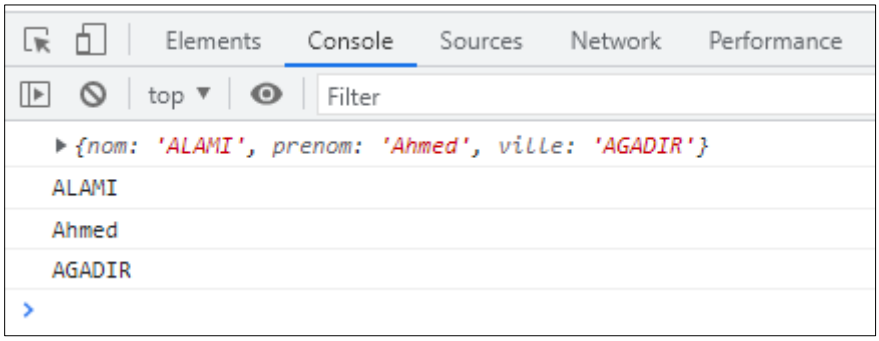
L'objet personne, intégré dans l'objet **personne2**, est devenu une propriété **personne** de ce nouvel objet. On aurait plutôt voulu que les propriétés de l'objet **personne** s'intègrent directement dans le nouvel objet, de façon à ce que celui-ci ait donc les propriétés **nom**, **prenom** et **ville** (au lieu des propriétés **personne** et **ville** comme c'est le cas actuellement).

L'opérateur **spread ...** va nous aider à réaliser l'éclatement des propriétés de l'objet personne en les intégrant directement dans le nouvel objet. Voici un exemple :

Exemple 2 :

```
<script>
        let personne = {nom : "ALAMI", prenom : "Ahmed"};
        let personne2 = {
            ...personne,
            ville : "AGADIR"
        };
        console.log(personne2);
        console.log(personne2.nom);
        console.log(personne2.prenom);
        console.log(personne2.ville);

</script>
```



L'exemple suivant montre l'utilisation de plusieurs opérateurs **spread ...** pour définir un nouvel objet :

Exemple 3 :

```
<script>
        let personne = {nom : "ALAMI", prenom : "Ahmed"};
        let place = {ville : "AGADIR", cp : 80000}            ;

        let contact = {...personne, ...place};
        console.log(contact);

</script>
```



8. Les tableaux

Nous venons de voir qu'il est possible de déstructurer ou structurer un objet. Ce type d'opération peut également être effectué avec les tableaux.

Déstructurer un tableau :

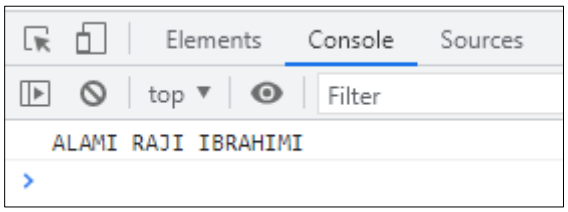
La déstructuration d'un tableau va permettre de récupérer des parties du tableau dans des variables.

Voici des exemples pour comprendre l'utilité de la destruction d'un tableau :

Exemple 1 :

```
<script>
        let noms = ["ALAMI", "RAJI", "IBRAHIMI"];;
        // Sans utiliser la destruction en ES6
        let nom1 = noms[0];
        let nom2 = noms[1];
        let nom3 = noms[2];
        console.log(nom1, nom2, nom3);

</script>
```

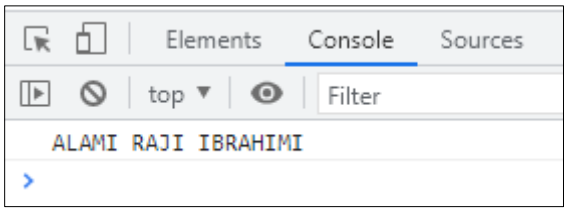


Exemple 2 :

```
<script>
        let noms = ["ALAMI", "RAJI", "IBRAHIMI"];;
        // Utiliser la destruction en ES6
        let [nom1, nom2, nom3] = noms;
        console.log(nom1, nom2, nom3);

</script>
```

Les éléments du tableau noms sont récupérés dans les variables nom1, nom2 et nom3, dans l'ordre où les éléments sont inscrits dans le tableau :

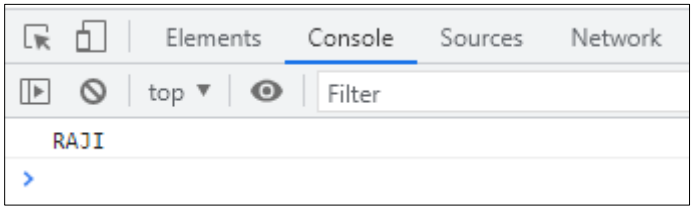


Si l'on souhaite uniquement récupérer le deuxième élément du tableau, on écrira le code suivant.

Exemple 3 :

```
<script>
        let noms = ["ALAMI", "RAJI", "IBRAHIMI"];;
        // Utiliser la destruction en ES6
        let [, nom2] = noms;
        console.log(nom2);

</script>
```

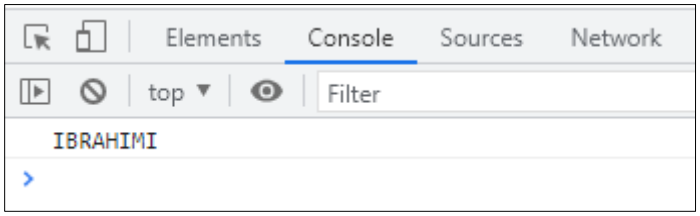


Si l'on souhaite uniquement récupérer le troisième élément du tableau, on écrira le code suivant.

Exemple 4 :

```
<script>
        let noms = ["ALAMI", "RAJI", "IBRAHIMI"];;
        // Utiliser la destruction en ES6
        let [, , nom3] = noms;
        console.log(nom3);

</script>
```



Structurer un tableau :

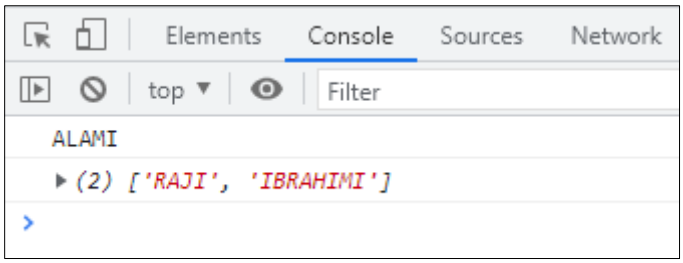
La structuration d'un tableau s'effectue principalement avec l'opérateur **spread ...** qui permet d'étendre les éléments d'un tableau.

Cet opérateur peut s'utiliser dans un tableau, afin de créer un nouveau tableau. Voici des exemples :

Exemple 1 :

```
<script>
        let noms = ["ALAMI", "RAJI", "IBRAHIMI"];;
        let [nom1, ...suite] = noms;
        console.log(nom1);
        console.log(suite);
</script>
```

On indique ici de créer les deux variables nom1 et suite, toutes deux extraites du tableau noms. La variable nom1 correspond au premier élément du tableau, tandis que la variable suite correspond au reste du tableau (après le premier élément déjà extrait) :

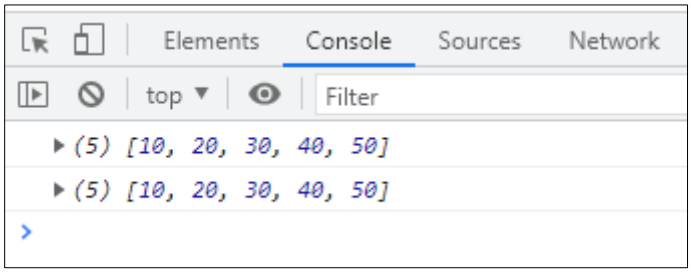


L'exemple suivant permet de créer une copie d'un tableau :

Exemple 2 :

```
<script>
        let numbers = [10, 20, 30, 40, 50]        ;
        let copy = [...numbers];
        console.log(numbers);
        console.log(copy);

</script>
```

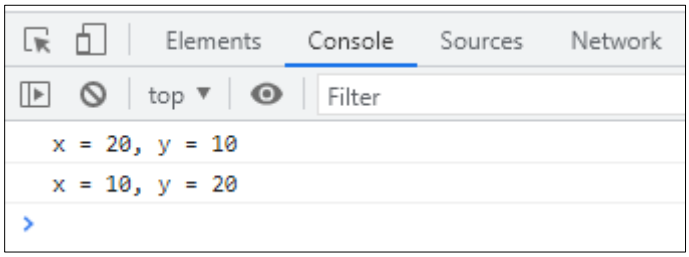


L'exemple suivant permet de permuter les valeurs de deux variables :

Exemple 3 :

```
<script>
        let x = 20;
        let y = 10;
        console.log(`x = ${x}, y = ${y}`);
        [x, y] = [y, x];
        console.log(`x = ${x}, y = ${y}`);

</script>
```



La méthode find :

La méthode **find()** renvoie le premier élément du tableau fourni qui satisfait la fonction de test fournie. Si aucune valeur ne satisfait la fonction de test, **undefined** est renvoyé.

Sa syntaxe : **array.find(function)**

Exemple 1 :

```
<script>
        let numbers = [11, 15, 22, 7, 9, 18];
        let even = n => n % 2 == 0;

        let result = numbers.find(even);
        console.log(result);     // 22
</script>
```

L'exemple précédent peut s'écrire de la façon suivante :

Exemple 2 :

```
<script>
        let numbers = [11, 15, 22, 7, 9, 18];

        let result = numbers.find(function(n){
            return n % 2 == 0;
        });
        console.log(result);                        // 22

        console.log(numbers.find(n => n % 2 != 0)); // 11

</script>
```

Voici un autre exemple :

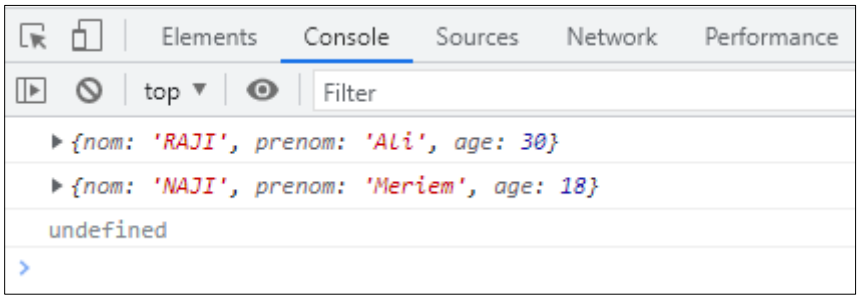
Exemple 3 :

```
<script>
        const personnes = [
            {nom : "ALAMI", prenom : "Ahmed", age : 22},
            {nom : "RAJI", prenom : "Ali", age : 30},
            {nom : "NAJI", prenom : "Meriem", age : 18},
            {nom : "RAMI", prenom : "Hicham", age : 18}
        ];
```

```
        const personne1 = personnes.find(per => per.age > 29 );
        const personne2 = personnes.find(per => per.age == 18 );
        const personne3 = personnes.find(per => per.age == 17 );

        console.log(personne1);
        console.log(personne2);
        console.log(personne3);

</script>
```



La méthode filter :

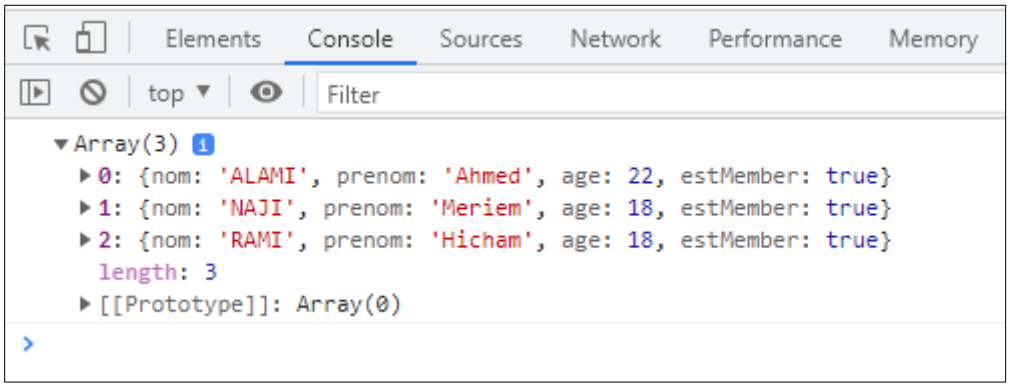
La méthode **filter()** crée une copie d'une partie d'un tableau donné, filtrée uniquement pour les éléments du tableau donné qui réussissent la condition implémentée par la fonction fournie.

Sa syntaxe : **array.filter(function)**

Exemple 1 :

```
<script>
        const personnes = [
            {nom : "ALAMI", prenom : "Ahmed", age : 22, estMember : true},
            {nom : "RAJI", prenom : "Ali", age : 30, estMember : false},
            {nom : "NAJI", prenom : "Meriem", age : 18, estMember : true},
            {nom : "RAMI", prenom : "Hicham", age : 18, estMember : true}
        ];
        let members = personnes.filter(per => per.estMember == true);
        console.log(members);


</script>
```



La méthode map :

La méthode **map()** est très populaire elle permet de :

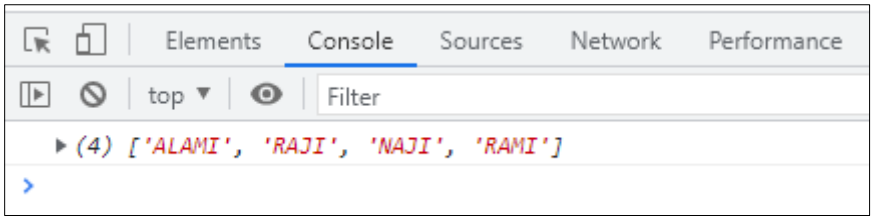
- Retourner un nouveau Array.
- Le nouveau Array a la même taille que l'Array d'origine.
- Utilise les valeurs de l'Array d'origine pour faire le nouveau Array.

Sa syntaxe : **array.map(function)**

Exemple 1 :

```
<script>
        const personnes = [
            {nom : "ALAMI", prenom : "Ahmed", age : 22, estMember : true},
            {nom : "RAJI", prenom : "Ali", age : 30, estMember : false},
            {nom : "NAJI", prenom : "Meriem", age : 18, estMember : true},
            {nom : "RAMI", prenom : "Hicham", age : 18, estMember : true}
        ];
        let noms = personnes.map(per => per.nom);
        console.log(noms);

</script>
```

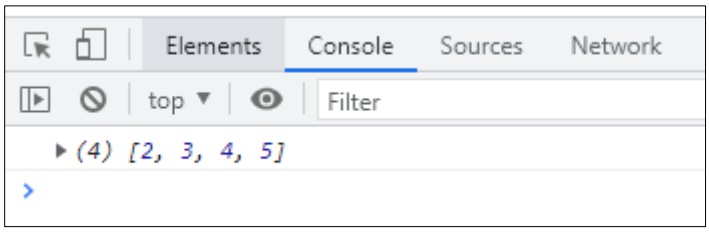


Exemple 2 :

```
<script>
	const numbers = [4, 9, 16, 25];
	let newArray = numbers.map(n => Math.sqrt(n));

	console.log(newArray);

</script>
```



La méthode reduce :

La méthode **reduce()** applique une fonction qui est qui traite chaque valeur du tableau (de la gauche vers la droite) afin de la réduire à une seule valeur.

Sa syntaxe : **array.reduce(function)**

Exemple 1 :

```
<script>
	const array1 = [1, 2, 3, 4];
	// 1 + 2 + 3 + 4
	let sum = array1.reduce((total, n) => total += n);
	console.log(sum); // 10

</script>
```

Exemple 2 :

```
<script>
	const employes = [
		{nom : "ALAMI", prenom : "Ahmed", salaire : 11000},
		{nom : "RAJI", prenom : "Ali", salaire : 5000},
		{nom : "NAJI", prenom : "Meriem", salaire : 7000},
		{nom : "RAMI", prenom : "Hicham", salaire : 15000}
	 ];
	 let sum = employes.reduce((total,emp) => total += emp.salaire,0);
	 console.log(sum);	// 38000

</script>
```

9. Les classes d'objets

Création d'une classe :

On utilise le nouveau mot-clé class (comme dans beaucoup d'autres langages). La création d'un objet de cette classe s'effectue au moyen de **new**.

L'exemple suivant permet de définir la classe **Personne** vide :

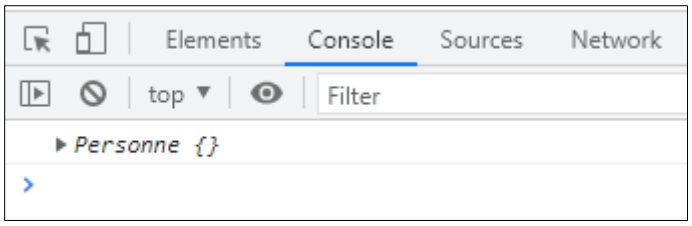
Exemple 1 :

```
<script>
        class Personne{

        }
        let personne = new Personne();

        console.log(personne);

</script>
```



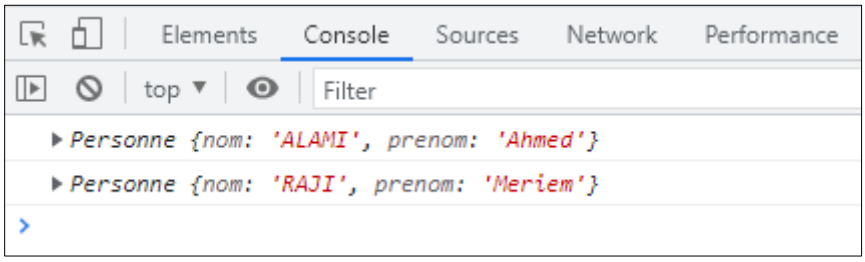
Pour définir des attributs dans une classe, il faut utiliser une méthode qui est appelée dès qu'un objet de cette classe est instancié. DansES6, cette méthode de construction s'appelle **constructor()** et aura en paramètres les éléments qui seront transmis lors de la création de l'objet par **new**. La méthode **constructor()** est présente dans toutes les classes que l'on crée et s'appelle le constructeur de la classe.

Exemple 2 :

```
<script>
        class Personne {
         constructor(nom, prenom){
             this.nom = nom;
             this.prenom = prenom;
         }
        }
        let personne1 = new Personne("ALAMI", "Ahmed");
        let personne2 = new Personne("RAJI", "Meriem");

        console.log(personne1);
        console.log(personne2);

</script>
```



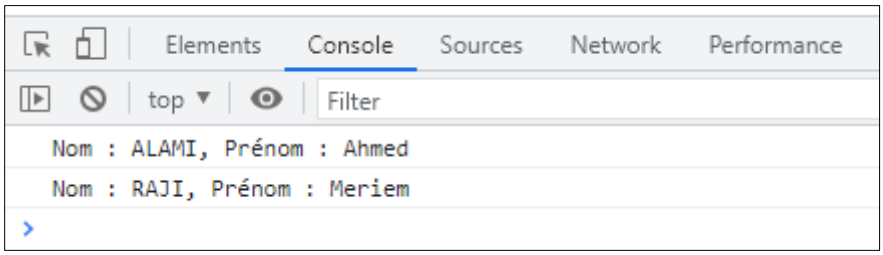
L'exemple suivant permet de créer la méthode **log()** dans la classe **Personne** :

Exemple 3 :

```
<script>
       class Personne {
        constructor(nom, prenom){
            this.nom = nom;
            this.prenom = prenom;
        }
        log(){
            console.log(`Nom : ${this.nom}, Prénom : ${this.prenom}`);
        }
       }
       let personne1 = new Personne("ALAMI", "Ahmed");
       personne1.log();

       let personne2 = new Personne("RAJI", "Meriem");
       personne2.log();

    </script>
```



Héritage de classe :

L'héritage de classe permet de créer une nouvelle classe à partir d'une classe déjà existante. On dit que la nouvelle classe est dérivée (héritée) de la première.

Pour illustrer cela, on souhaite créer une nouvelle classe, appelée **Homme**, dérivée de la classe **Personne** écrite précédemment. On peut de la même façon créer la classe **Femme**, également dérivée de la classe **Personne**, mais qui permet de traiter les personnes de sexe féminin.

Pour indiquer qu'une classe dérive d'une autre classe, on utilise le mot-clé **extends**. Ainsi, pour indiquer que la classe Homme dérive de la classe Personne, on écrira ce qui suit.

Il faut également appeler le constructeur de la classe Personne (classe parente), ce qui se fait en utilisant la méthode **super()** à laquelle on transmet les arguments **nom** et **prenom** qui définissent un objet Personne.

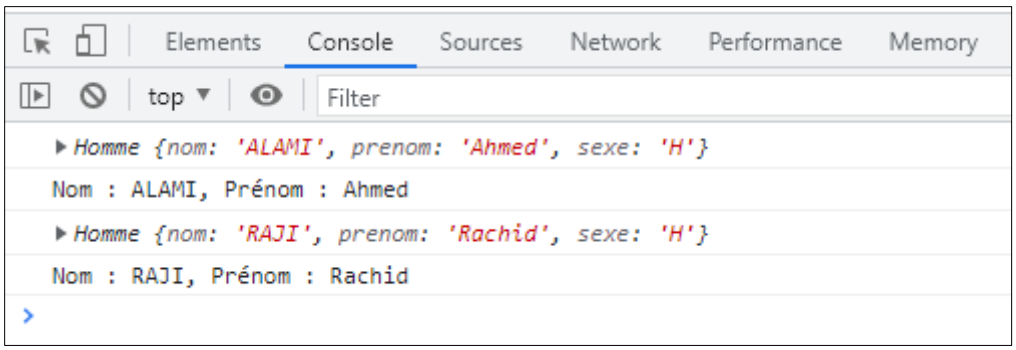
Exemple 1 :

```
<script>
        class Personne {
         constructor(nom, prenom){
             this.nom = nom;
             this.prenom = prenom;
         }
         log(){
             console.log(`Nom : ${this.nom}, Prénom : ${this.prenom}`);
         }
        }

        class Homme extends Personne {
         constructor(nom, prenom){
             super(nom, prenom);
             this.sexe = "H";
         }
        }
        let personne1 = new Homme("ALAMI", "Ahmed");
        console.log(personne1);
        personne1.log();

        let personne2 = new Homme("RAJI", "Rachid");
        console.log(personne2);
        personne2.log();

    </script>
```



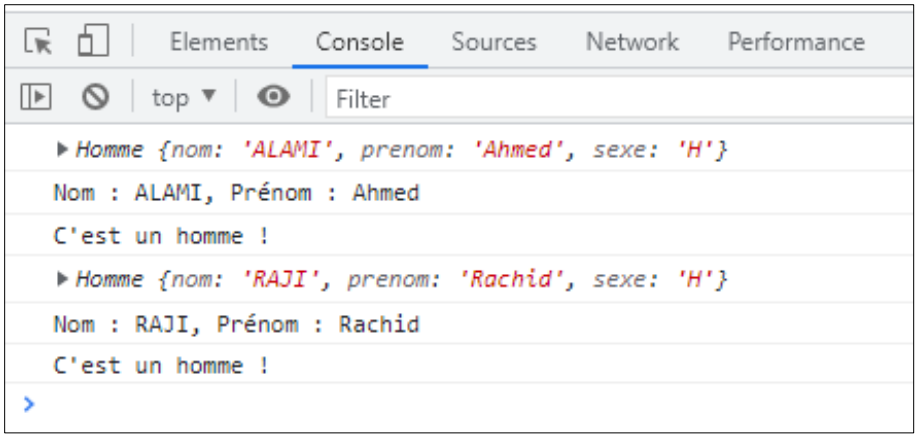
L'exemple suivant permet d'ajouter une nouvelle méthode **log()** dans la classe **Homme** :

Exemple 2 :

```
<script>
      class Personne {
       constructor(nom, prenom){
           this.nom = nom;
           this.prenom = prenom;
       }
       log(){
           console.log(`Nom : ${this.nom}, Prénom : ${this.prenom}`);
       }
      }

      class Homme extends Personne {
       constructor(nom, prenom){
           super(nom, prenom);
           this.sexe = "H";
       }
       log(){
           // Appel de la méthode log() définie dans la classe parente
           super.log();
           console.log("C'est un homme !");
       }
      }
      let personne1 = new Homme("ALAMI", "Ahmed");
      console.log(personne1);
      personne1.log();

      let personne2 = new Homme("RAJI", "Rachid");
      console.log(personne2);
      personne2.log();
   </script>
```



10. Les modules

ES6 permet de définir des modules, c'est-à-dire des fichiers JavaScript qui permettent de diviser un programme JavaScript en fichiers autonomes, inclus dans la page HTML ou dans d'autres modules.

Un module permet de définir des constantes, des variables, des fonctions ou des classes (en fait, tout type de donnée) qui sont par défaut privées pour le module (les autres modules n'y ont pas accès, à moins de les y autoriser).

Pour bien comprendre l'apport des modules ES6 par rapport aux fichiers JavaScript traditionnels, divisons le fichier HTML précédent en divers fichiers utilisés ensemble.

Division d'une page HTML en plusieurs fichiers :

Créons un fichier personne.js permettant de définir la classe Personne vue précédemment. Pour l'instant, on ne constate pas de différence entre un fichier JavaScript classique et un module ES6.

Fichier personne.js définissant la classe Personne :

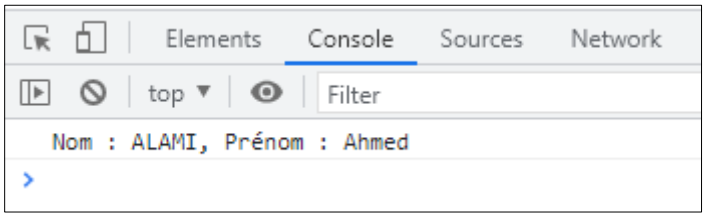
```
class Personne {
    constructor(nom, prenom){
        this.nom = nom;
        this.prenom = prenom;
    }
    log(){
        console.log(`Nom : ${this.nom}, Prénom : ${this.prenom}`);
    }
   }
```

La classe **Personne** peut être utilisée dans un fichier HTML en incluant le fichier **personne.js** au moyen d'une balise <**script**> classique :

Exemple 1 :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <script src="./personne.js" ></script>
    <title>Document</title>
</head>
<body>

    <script>
           let personne1 = new Personne("ALAMI", "Ahmed")          ;
           personne1.log();
     </script>
</body>
</html>
```



Faisons la même chose avec la classe Homme, en la définissant dans un fichier **homme.js** qui sera lui aussi inclus dans la page HTML :

Fichier homme.js définissant la classe Homme :

```
class Homme extends Personne {
    constructor(nom, prenom){
        super(nom, prenom);
        this.sexe = "H";
    }
    log(){
        // Appel de la méthode log() définie dans la classe parente
        super.log();
        console.log("C'est un homme !");
    }
   }
```

Pour inclure ce fichier dans la page HTML, il est obligatoire d'inclure au préalable le fichier **personne.js** qui définit la classe **Personne** (sinon la classe Personne, utilisée par la classe **Homme**, est inconnue) :

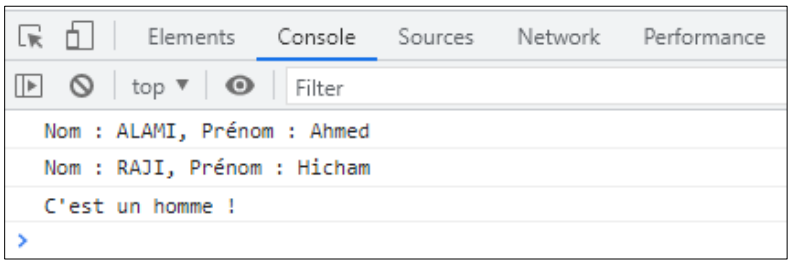
Exemple 2 :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <script src="./personne.js" ></script>
    <script src="./homme.js" ></script>
    <title>Document</title>
</head>
<body>

    <script>
           let personne1 = new Personne("ALAMI", "Ahmed")           ;
           personne1.log();

           let personne2 = new Homme("RAJI", "Hicham");
           personne2.log();
     </script>
</body>
</html>
```

Les deux fichiers JavaScript incluant les classes Personne et Homme sont inclus, ce qui permet d'utiliser ces deux classes dans le script de la page HTML :



Intérêt des modules :

Dans l'exemple précédent, nous n'avons pas utilisé les modules tels que ES6 les définit. Nous avons simplement découpé un fichier en plusieurs fichiers, inclus ensuite dans le fichier principal.

Cette approche, bien que fonctionnelle, présente un inconvénient majeur (dans le cadre d'un développement nécessitant beaucoup de fichiers). En effet, lors de l'inclusion des fichiers JavaScript nécessaires dans le fichier principal :

- L'ordre d'inclusion doit être connu : il faut inclure personne.js avant homme.js, sinon cela ne fonctionne pas ;
- Les dépendances entre fichiers doivent être connues : le fichier homme.js nécessite d'inclure le fichier personne.js, sinon la classe Homme ne peut pas être créée à partir de la classe Personne.

Le but des modules ES6 est d'aider à utiliser ces fichiers (ici, personne.js et homme.js) sans avoir à connaître ces informations. En effet, ces dernières seront insérées sous une certaine forme dans le fichier lui-même, qui devient ainsi un module.

Export de données :

Dans chaque module (en fait, un fichier contenant du code JavaScript), on décide quelles sont les données que l'on souhaite rendre visibles à l'extérieur (pour d'autres modules ou pour une page HTML). On utilise pour cela le nouveau mot-clé JavaScript **export**, qui permet d'exporter une ou plusieurs données vers l'extérieur.

Dans notre exemple, on souhaite exporter les classes Personne et Homme de façon à les rendre accessibles à l'extérieur. On ajoute alors le mot-clé **export** dans chacun des deux fichiers (qui deviennent ainsi des modules).

Module personne.js

```
class Personne {
    constructor(nom, prenom){
        this.nom = nom;
        this.prenom = prenom;
    }
    log(){
        console.log(`Nom : ${this.nom}, Prénom : ${this.prenom}`);
    }
   }
export {Personne};
```

Ainsi, pour transformer un fichier JavaScript en module, il suffit de lui ajouter le mot-clé **export** précisant les données exportées, séparées par des virgules si plusieurs données sont à exporter, et entourées d'accolades : **export {data1, data2, ...} ;**

Si une seule donnée est exportée dans le module, on peut écrire également la ligne suivante :

export default data ;

Dans l'exemple précédent du module personne.js on peut écrire : **export default Personne ;**

Utilisation du module dans le fichier HTML :

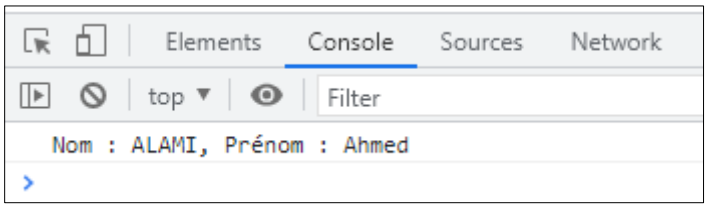
Le fichier JavaScript du module est inclus au moyen de l'instruction import. Pour que cette instruction soit comprise par le navigateur, il faut qu'elle soit insérée dans une balise **<script type="module">**, sinon l'instruction import n'est pas reconnue.

Exemple 1 :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Document</title>
</head>
<body>

    <script type="module">
           import {Personne} from './personne.js'
           let personne1 = new Personne("ALAMI", "Ahmed")          ;
           personne1.log();

     </script>
</body>
</html>
```



Import de données :

Dans l'exemple précédent, nous avons utilisé l'instruction import afin que notre script puisse utiliser les données du module importé. Les données utilisées dans le script (et importées depuis le module) sont précisées dans l'instruction import.

On ne peut importer que des données qui ont été précédemment exportées, sinon une erreur se produit. Par exemple, il est impossible d'importer la classe Personne si cette classe n'a pas été exportée dans le module où elle est définie.

La syntaxe d'importation d'un module : **import {data1, data2, ...} from 'file.js' ;**

Et pour importer une donnée exportée par **export default** : **import data1 from 'file.js' ;**

Exemple 1 :

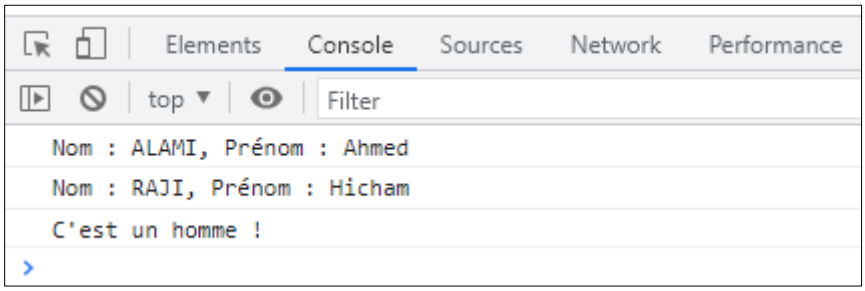
Module homme.js

```
import {Personne} from './personne.js'
class Homme extends Personne {
    constructor(nom, prenom){
        super(nom, prenom);
        this.sexe = "H";
    }
    log(){
        // Appel de la méthode log() définie dans la classe parente
        super.log();
        console.log("C'est un homme !");
    }
   }
export default Homme;
```

La page index.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Document</title>
</head>
<body>
    <script type="module">
            import {Personne} from './personne.js';
            import Homme from './homme.js'
            let personne1 = new Personne("ALAMI", "Ahmed")          ;
            personne1.log();

            let personne2 = new Homme("RAJI", "Hicham");
            personne2.log();
     </script>
</body>
</html>
```



Activité N°1 :

Vous trouverez ci-joint le **PDF** :

PDF : Activite1.pdf